

Tipos

João Eduardo Montandon
DCC024

Introdução

- Em quantas linguagens você já programou?
 - Em qual delas você **não precisou** trabalhar com tipos? 🤔
 - Intuitivamente conhecemos as regras básicas que cercam os tipos...
-

🎯 O que são tipos? Para que são utilizados? Por que existem?

Um Tipo É Um Conjunto

```
int n;
```



"A variável `n` pode receber qualquer valor do conjunto **inteiro**"

Um tipo representa um conjunto de valores

- Todos os elementos representados por aquele tipo são codificados em memória da mesma forma
 - Todos os elementos estão submetidos as mesmas operações para aquele tipo
-

Se Tipos São Conjuntos

Nós podemos manipulá-los e operá-los como conjuntos!

Anatomia Dos Tipos

1. Tipos Primitivos
2. Tipos Construídos

Tipos Primitivos

Qualquer tipo que pode ser utilizado, mas não definido no programa.

- Não podem ser replicados!
- ML: `int`, `real`, `char`
- Não é possível criar um tipo `int` que funcione como `int`

-
- Especificados na definição da linguagem
 - Algumas apresentam definições mais rigorosas do que outras
 -  Java (`int` tem 32 bits com sinal, sempre!)
 -  C, ML (Mais espaço para `int` conforme hardware)

Tipos Construídos

Qualquer tipo que pode ser definido pelo programa

- ML: `type ponto = int * int;`
 - Tipo `ponto`, representado por tupla de inteiros

Diferentes formas de se definir tipos construídos, **matematicamente!**

1. Enumeração
 2. Tuplas
 3. Vetores, listas, strings
 4. Uniões
 5. Subtipos
 6. Funções
-

Enumeração

$$S = \{a, b, c\}$$

- Forma mais simples para construção de tipos
- Tipo é definido a partir de um conjunto de valores pré-definidos

```
enum coin {penny, nickel, dime, quarter};
```

```
datatype dia = Dom | Seg | Ter | Qua | Qui | Sex | Sab;
```

Podem ser tratados como valores de outros tipos:

```
enum coin {penny=1, nickel=5, dime=10, quarter=25};  
int trocados = penny + nickel + dime; // 1 + 5 + 10 = 16
```

Ou possuírem seu valor próprio:

```
- fun ehFds d = (d = Sab orelse d = Dom);  
- ehFds Seg;  
val it = false : bool;
```

Tuplas

$$\begin{aligned} S &= X \times Y \\ &= \{(x, y) | x \in X \wedge y \in Y\} \end{aligned}$$

- Uma tupla é uma combinação (produto cartesiano) dos elementos presentes nos tipos (conjuntos) que a compõem

$$\{1, 2\} \times \{r, g, b\} = \{(1, r), (2, r), (3, r), (1, g), (2, g), (3, g), (1, b), (2, b), (3, b)\}$$

Algumas linguagens suportam tuplas puras:

```
type coord = real * real;
fun getLat (x: coord) = #1 c;
```

Outras linguagens suportam tuplas nomeadas (a.k.a *structures*, *records*)

```
type coord = {
    lat: real,
    long: real
};
fun getLat (c : coord) = #lat c;
```

Representação

- Normalmente, elementos da tupla são colocados lado-a-lado em memória
- Mas...
 - Em qual ordem?
 - Como os elementos são separados?

O desenvolvedor tem gerência sobre algumas dessas perguntas? 🤔

Exemplo: ANSI C

If a pointer to a structure is cast to the type of a pointer to its first member, the result refers to the first member...

-- The C Programming Language, 2nd ed.

Exemplo: ML

- Representação de tuplas totalmente encapsulada
- Única operação disponível: `#pos tupla`

Vetores

Tamanho fixo:

$$\begin{aligned} S &= X^n \\ &= \{(x_1, \dots, x_n) \mid \forall i. x_i \in X\} \end{aligned}$$

Tamanho variável:

$$\begin{aligned} S &= X^* \\ &= \bigcup_i X^i \end{aligned}$$

-
- Se aplica a vetores, strings, e listas
 - Mesmo princípio de tuplas ($A \times A \times A \times \dots A$)
 - Porém possuem maior flexibilidade
-

Exemplo: Índices

- Java, C, C++:
 - Primeiro elemento do vetor `a` é `a[0]`
 - Índices sempre inteiros
 - Pascal:
 - Pode utilizar outros tipos como índices (inteiros, caracteres, enums, etc)
 - Valores inicial e final são escolhidos pelo desenvolvedor
 - Tamanho é definido em nível de compilação
-

```
type
    LetterCount = array['a'..'z'] of Integer;
var
    Counts: LetterCount;

begin
    Counts['a'] = 1
    ...
```

-
- Quais valores podem ser utilizados como índice
 - O tamanho é definido durante compilação ou execução?
 - Quais operações podem ser feitas?
 - Pode ser redimensionado?
 - Suporta múltiplas dimensões?
 - Qual a ordem dos elementos em memória?
 - Existe um tipo próprio para strings?

- Existe um tipo próprio para listas?

Não existe uma resposta definitiva para essas perguntas! São decisões de design e implementação.

Unões

$$S = X \cup Y$$

- Conjunto de todos elementos do tipo X ou Y (ou ambos)
 - Suportado em muitas linguagens de programação
 - Um pouco mais complicado do que parece
 - Como saber qual tipo do valor armazenado?
 - Quais operações podem ser feitas neste tipo?
-

Exemplo: ANSI C

```
union element {  
    int i;  
    float f;  
};  
  
union element e;  
e.i = 100;  
float x = e.f;  
printf("%f", x); // o que vai ser impresso?
```

- Espaço na memória: `max(sizeof(e.i), sizeof(e.f))`
 - Não é possível saber qual o tipo do valor armazenado ⚠
-

Exemplo: ML

```
datatype element = I of int | F of real;
```

```
fun getReal (F x) = x
  | getReal (I x) = real x;
```

- Todo `element` é acompanhado do tipo do valor que ele armazena
- É preciso especificar o comportamento para extrair o valor para cada situação

Subtipos

$$S = \{x \in Y | Y \subset X\}$$

- Tipos construídos para possuir um subconjunto de elementos de um tipo já existente
- Suportado em diferentes formas
 - Pascal: `type digit = 0..9;`
 - Ada: `subtype DIGIT is INTEGER range = 0..9;`
 - Lisp: `(declare (type (and integer (satisfies evenp)) x))`

Representação Dos Valores

- Geralmente utilizamos a mesma representação do supertipo
- Mas é possível otimizar essa representação, afinal um subtipo sempre tem menos elementos que seu supertipo
 - Exemplo: `X: 1..9` deve ocupar mesmo que `X: Integer`?

Operações

- Permitidas todas as operações do supertipo
- Suporta operações específicas para o subtipo
 - `function toChar(X: Digit): Char;`

Um subtipo é um subconjunto de valores, mas suporta um superconjunto de operações

Soa familiar? 🤔

Funções

$$\begin{aligned} S &= D \rightarrow I \\ &= \{f \mid \text{dom } f = D \wedge \text{im } f = I\} \end{aligned}$$

Noção de funções é similar na maioria das linguagens

```
int f(char a, char b) {  
    return a == b;  
}
```

```
fun f(a : char, b : char) = (a == b);
```



```
char * char -> bool
```

Operações Com Funções

Se funções são tipos, e tipos são associados a variáveis...

1. Podemos armazenar uma função em uma variável?
2. Podemos criar um vetor de funções?
3. Podemos passar uma função de parâmetro?
4. Podemos retornar uma função?
5. Existem operadores para funções?

Que tipo de linguagem suporta esses recursos? Acho que você já conhece...^[1] 🤖

Uso Para Tipos

Utilizados de muitas formas em diferentes linguagens

1. Anotação de Tipos
2. Inferência de Tipos
3. Checagem de Tipos

Anotação De Tipos

```
int idade;
```

Permite definir o tipo associado aos membros do programa (variáveis, funções, ...)

- 🎯 Fornecer informações de tipo durante o processo de compilação
 - 👍 Ferramenta de documentação
 - Diferentes estratégias adotadas por diferentes linguagens
 - Java: obrigatório para todas as variáveis
 - ML: opcional
 - Prolog: não é permitido
-

Inferência De Tipos

```
- fun resto (a, b) = a mod b;  
val resto = fn : int * int -> int
```

Obtenção do tipo através de informações fornecidas por constantes, operadores, etc

- Constantes geralmente possuem tipos estáticos
 - `10` é `int`, enquanto `10L` é `long` (Java)
 - Expressões podem inferir seu tipo a partir de seus operandos
 - se `a` é `double`, então `a * 0` é `double` (`0.0`)
 - Algumas linguagens levam esse processo ao extremo, e procuram inferir o tipo de **toda** expressão (ML)
-

Checagem De Tipos

```
not x
```

E se o tipo de `x` for uma `string`? 🤔

É preciso checar se há consistência entre o tipo do valor e a operação na qual ele é submetido

- Realizado por praticamente toda a linguagem de programação
 - Pode ser realizada estaticamente ou dinamicamente
 - Maioria das linguagens modernas ➡ checagem estática
-

Checagem Estática

- Determina o tipo de todos os membros **antes** do programa executar
 - variáveis, funções, expressões, etc
- Erros de compilação são exibidos quando tipos não são consistentes
- Linguagens conhecidas como **estaticamente tipadas**

```
1 + "abc"; ✖  
round("abc"); ✖  
if "abc" then ... ✖
```

Checagem Dinâmica

- Checagem não é feita antes do programa ser executado
- Em *runtime*, a linguagem checa se operandos são compatíveis com os operadores da instrução
- Linguagens conhecidas como **dinamicamente tipadas**

```
(defun f (a) (+ a nil)) ; Operação incompatível, mas não há erro!  
...  
(f 10) ; Erro é lançado apenas aqui!
```

Tipagem Dinâmica ≠ Tipagem Explícita

- Algumas linguagens não anotam tipos, mas mesmo assim realizam a checagem estática (ML)
- Lembre-se, tipos podem ser obtidos a partir da [Anotação](#) ou [Inferência](#)

```
- fun negacao a : string = not a;  
Error: right-hand-side of clause does not agree with function result type
```

Esse erro aconteceu em tempo de compilação ou execução? 😞

Tipagem Forte X Tipagem Fraca

- O propósito da checagem de tipos é prevenir que o programa execute operações com tipos incorretos em seus operandos
- Quando a checagem é extensiva para garantir que esses erros não ocorram (Java, ML): **Tipagem Forte**
- Quando a checagem não fornece essa garantia (C): **Tipagem Fraca**

Mesmo assim, não é 8 ou 80!

- Algumas checagens são feitas apenas durante a execução do programa
- Exemplo: subtipos!

Equivalência De Tipos

```
type irpar1 = int * real;  
type irpar2 = int * real;  
fun f(x : irpar1) = #1 x;  
  
val v1 = (1, 2.0) : irpar1;  
val v2 = (1, 2.0) : irpar2;  
  
f v1; (* Funciona? *)  
f v2; (* Funciona???? 😞 *)
```

-
- Checagem de tipos: verificar se o tipo das variáveis são compatíveis para a operação
 - Mas e se o programa não conseguir definir o tipo?
 - Diferentes linguagens adotam diferentes estratégias
-

1. **Equivalência por nome:** tipos são equivalentes se eles possuem o mesmo nome
 - Exemplo: Pascal
 2. **Equivalência por estrutura:** tipos são equivalentes se foram construídos da mesma forma
 - Exemplo: ML
-

Conclusões

Um tipo é um conjunto

- Tipos primitivos: presentes na linguagem, não podem ser replicados
 - Tipos construídos: tipos criados a partir dos tipos primitivos
 - Diferentes linguagens adotam diferentes estratégias para construir e manipular esses tipos
 - C: mais transparente e flexível, mas perigoso
 - ML: mais abstrata e opaca, mas possui menos especificidades.
-
-

1. [First-class function - Wikipedia](#) ↩